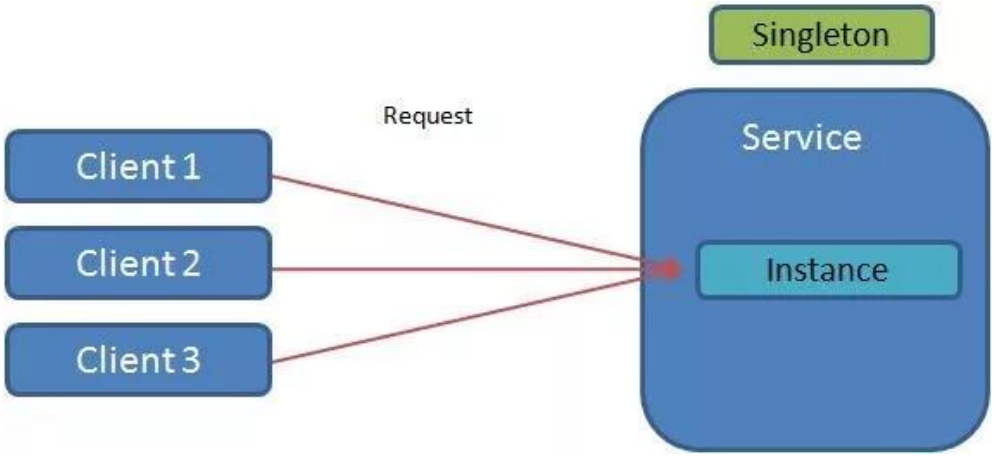
Порождающие паттерны. Одиночка

Паттерн Одиночка
Singleton



Одиночка — это порождающий паттерн проектирования, который гарантирует, что у класса есть только один экземпляр, и предоставляет к нему глобальную точку доступа.

Проблема

Одиночка решает две проблемы, нарушая *принцип единственной ответственности* класса.

1. Гарантирует наличие единственного экземпляра класса.

Чаще всего это полезно для доступа к какому-то общему ресурсу, например, базе данных.

Такое поведение невозможно реализовать с помощью обычного конструктора, так как конструктор класса **всегда** возвращает новый объект.

2. Предоставляет глобальную точку доступа к экземпляру класса.

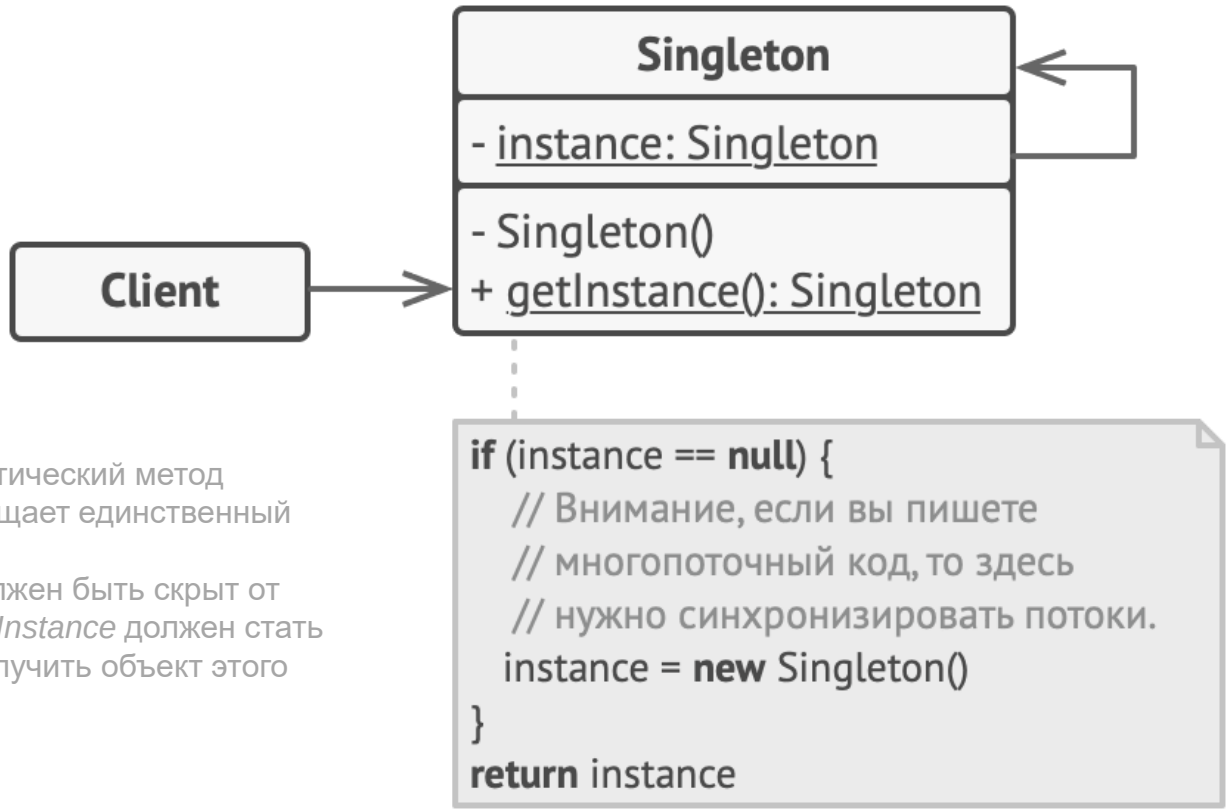
Это должна быть не просто глобальная переменная, через которую можно обраться к определённому объекту.

Глобальные переменные не защищены от записи, поэтому любой код может подменять их значения без ведома владельца.

Решение

- Все реализации **Одиночки** сводятся к тому, чтобы скрыть *конструктор по умолчанию* и создать *публичный статический метод*, который и будет контролировать жизненный цикл *объекта-одиночки*.
- Если у клиента есть доступ к классу **Одиночки**, значит, будет доступ и к этому статическому методу. При вызове этого метода, он всегда будет отдавать один и тот же объект.

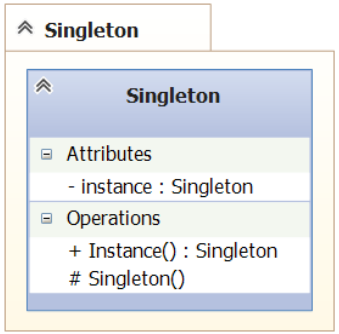
Структура паттерна



[1]**Одиночка** определяет статический метод getInstance, который возвращает единственный экземпляр своего класса.

Конструктор одиночки должен быть скрыт от клиентов. Вызов метода *getInstance* должен стать единственным способом получить объект этого класса.

Структура паттерна на языке C#




```
class Program
{
    static void Main()
    {
        Singleton instance1 = Singleton.Instance();
        Singleton instance2 = Singleton.Instance();
        Console.WriteLine(ReferenceEquals(instance1, instance2));

        instance1.SingletonOperation();
        string singletonData = instance1.GetSingletonData();
        Console.WriteLine(singletonData);
    }
}
```

```
class Singleton
{
    static Singleton uniqueInstance;
    string singletonData = string.Empty;

    protected Singleton()
    {
    }

    public static Singleton Instance()
    {
        if (uniqueInstance == null)
            uniqueInstance = new Singleton();

        return uniqueInstance;
    }

    public void SingletonOperation()
    {
        singletonData = "SingletonData";
    }

    public string GetSingletonData()
    {
        return singletonData;
    }
}
```

Участники

Singleton - **Одиночка**:

Предоставляет интерфейс (метод Instance) для получения доступа к единственному экземпляру.

Отношения между участниками

Отношения между классами

Класс Singleton не имеет обязательных связей отношений.

Отношения между объектами

Клиент получает доступ к экземпляру через вызов метода Instance() объекта класса Singleton.

Применение

1. Когда в программе должен быть единственный экземпляр какого-то класса, доступный всем клиентам (например, общий доступ к базе данных из разных частей программы).

Одиночка скрывает от клиентов все способы создания нового объекта, кроме специального метода. Этот метод либо создаёт объект, либо отдаёт существующий объект, если он уже был создан.

2. Когда необходимо иметь больше контроля над глобальными переменными.

В отличие от глобальных переменных, **Одиночка** гарантирует, что никакой другой код не заменит созданный экземпляр класса, т.е. гарантирует наличие лишь одного объекта-одиночки.

Тем не менее, можно расширить это ограничение и позволить любое количество объектов-одиночек, поменяв код в одном месте (метод *getInstance*).

Шаги реализации

1. **Добавить в класс приватное статическое поле**, которое будет содержать одиночный объект.
2. **Объявить статический создающий метод**, который будет использоваться для получения одиночки.
3. **Добавить *«ленивую инициализацию»*** (или *отложенная* - создание объекта в момент обращения к нему) в создающий метод одиночки.
4. **Сделать конструктор класса *приватным***.
5. **В клиентском коде замените вызовы конструктора одиночки** вызовами его создающего метода.

Преимущества и недостатки

+ Гарантирует наличие единственного экземпляра класса.

+ Предоставляет к нему глобальную точку доступа.

+ Реализует *отложенную («ленивую»)* инициализацию объекта-одиночки.

- Нарушает принцип единственной ответственности класса.

- Маскирует плохой проект (нарушение принципа единственной ответственности).

- Проблемы многопоточности (*гонки потоков* при создании экземпляра одиночки).

-Требует постоянного создания Mock-объектов («заглушка» интерфейса) при юнит-тестировании. Не получится использовать класс, зависящий от одиночки в другой программе. Для этого придётся эмулировать присутствие одиночки и там.